

2. Forward 와 Redirect

2. Dispatch, Forward, Redirect

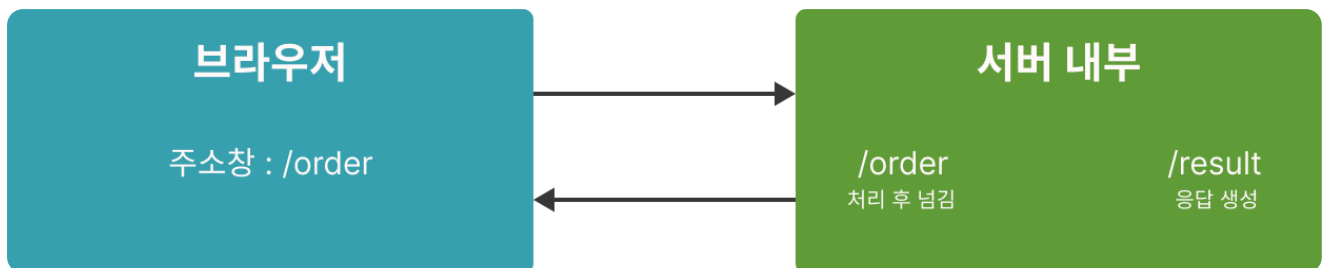
본격적으로 Filter 에 들어가기 전에 Dispatch, Forward, Redirect 에 대해 간략하게 짚고 넘어가고자 한다.

들어는 보았어도 명확하게 알지 못했던 개념이라 이번 기회에 정리해 보았다.

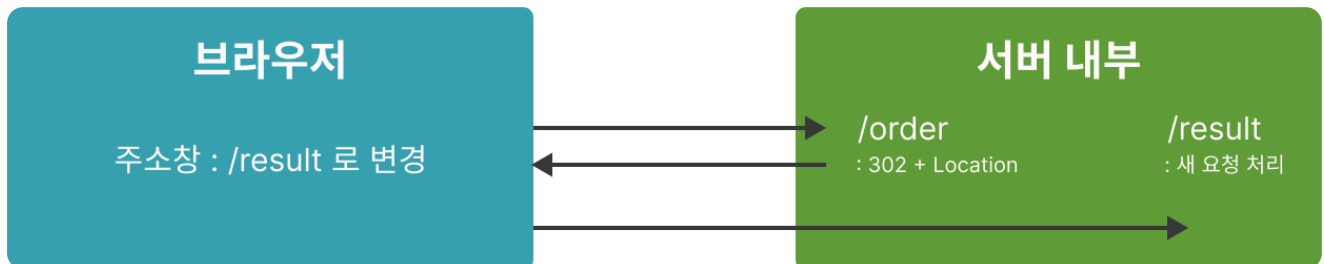
2.1 Forward 와 Redirect

- **Forward** 는 서버 내부에서 처리를 넘기는 것이고
- **Redirect** 는 클라이언트에게 다시 요청하라고 시키는 것이다

Forward



Redirect



예제에서 보듯 Forward 는 요청이 1회이고, Redirect 는 요청이 2회이다.

포워드에서 브라우저는 아무것도 모른다.

요청 하나를 보내고 응답 하나를 받았을 뿐이며, 그 안에서 서버가 처리를 넘겼는지는 알 길이 없다.

반면 리다이렉트에서는 브라우저가 두 번 움직인다.

2.2 그래서 정체는 무엇이나

Forward 는 Servlet API 의 기능이다.

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/result");  
dispatcher.forward(request, response);
```

`HttpServletRequest` 와 `HttpServletResponse` 객체를 그대로 들고 다른 서블릿 혹은 JSP 로 넘긴다.

같은 스레드, 같은 요청 객체이며 네트워크는 개입하지 않는다.

브라우저의 주소창도 그대로 유지된다.

Redirect 는 HTTP 프로토콜의 기능이다.

```
response.sendRedirect("/result");
```

```
// HTTP/1.1 302 Found
```

```
// Location: /result
```

본문 없이 상태 코드와 `Location` 헤더만 있는 응답이다.

브라우저가 이것을 보고 `Location` 의 주소로 새 요청을 자동으로 보낸다.

따라서 주소창의 URL 도 새 주소로 바뀐다.

	포워드	리다이렉트
요청 횟수	1회	2회
주소창 URL	그대로	목적지로 바뀜
request 객체	공유됨	새로 생성
request 속성(attribute)	유지됨	사라짐
외부 도메인	불가	가능
클라이언트 인지	못 함	함
네트워크 왕복	없음	1회 추가
브라우저 뒤로가기	이전 페이지로	목적지 이전으로

`request` 속성의 공유 여부는 실무에서 자주 체감하는 부분이다.

포워드는 `request.setAttribute()` 로 담은 데이터가 그대로 넘어가지만, 리다이렉트는 **요청 자체가 새 것**이라 전부 날아간다.

그래서 리다이렉트로 데이터를 넘기려면 쿼리 파라미터에 실어 보내거나 세션을 거쳐야 한다.

2.2 Dispatch

일상어로 **dispatch**는 보내다, 배정하다 라는 뜻을 가진다.

ex)

- 119 상황실에서 신고를 받고 구급차를 배정하는 것
 - 콜센터에서 접수 건을 담당자에게 넘기는 것
- > dispatch

그 일을 하는 사람 -> dispatcher

Servlet에서는 컨테이너(톰캣)가 요청 받아서 처리할 컴포넌트에게 넘기는 행위이다.

2.2.1 HTTP 요청 1건 = 디스패치 1회 ?

하나의 HTTP 요청이 여러 번의 디스패치를 일으킬 수 있다.

그리고 디스패치 일어날 때마다 필터 체인을 다시 통과한다

ex

[HTTP 요청 1건 : 브라우저가 보낸 것은 하나]

1) REQUEST(디스패치) → 필터체인(통과) → 컨트롤러 실행 (뷰 이름 반환)

-- 뷰 렌더링을 위해 forward 발생 --> 2) FORWARD (디스패치) → 필터체인 (기본 매핑 X)
-> 렌더링 (응답 완성)

브라우저는 요청을 한 번 보냈지만 Tomcat 안에서는 처리 사이클이 두 번 돈다
디스패치는 이 사이클 하나를 가리킨다.

2.2.2 DispatcherType

DispatcherType 은 요청 종류가 아니라 **디스패치 발생 원인을 분류한 값**이다.

- **REQUEST** : 클라이언트 요청이 처음 들어옴
- **FORWARD** : RequestDispatcher.forward() 호출
- **INCLUDE** : RequestDispatcher.include() 호출
- **ASYNC** : 비동기 처리가 끝나고 재개됨
- **ERROR** : 예외 발생으로 에러 페이지로 넘어감

Redirect 는 서버 밖으로 나갔다가 새 HTTP 요청으로 들어오는 것이어서 새로운 **REQUEST** 일 뿐이다.
반면 **Forward** 는 서버 안에서 사이클만 더 도는 것이라 별도의 타입이 필요하다.

INCLUDE 는 포워드가 제어권 넘기는 것이라면 **INCLUDE** 는 다른 리소스의 출력을 현재 응답 중간에 끼워넣고 돌아오는 것이다.

필터는 디스패치마다 실행된다

그래서 기본 값이 **REQUEST** 인데, 포워드에도 매핑된다면 인증 필터가 한 요청에 두 번 돌게 된다.
이 때 **OncePerRequestFilter** 라는 것이 디스패치마다 기본 동작이 이루어지는 것을 HTTP 요청 당 한 번을 보장해준다.

2.2.3 DispatcherServlet 의 dispatch 는 다른 층이다.

서블릿 컨테이너의 디스패치

- 주체는 톰켓이며 필터 체인에서 서블릿으로 넘긴다

DispatcherServlet의 디스패치

- 주체가 스프링이며 핸들러 매핑에서 컨트롤러로 넘긴다

DispatcherServlet 은 그 자체가 하나의 서블릿이다.

톰켓이 REQUEST 디스패치로 이 서블릿을 호출하면, 그 안에서 다시 URL 을 보고 적절한 컨트롤러로 배정한다.

바깥 디스패치 안에 안 쪽 디스패치가 들어있는 구조다.